

Assignment 5

1. Assignment Description

Problem: 2D Ray Tracing with QuadTree Acceleration (50 points)

In computer graphics and physical simulations, checking collisions or intersections in a massive continuous space can cause severe performance bottlenecks. Specifically, we will explore **2D Ray Tracing**, a rendering technique that models the physical paths of light. While real-world ray tracing operates in complete 3D environments and simulates complex optical phenomena such as refraction and scattering, this assignment simplifies the problem to a 2D space focusing solely on basic intersections and reflections. Ray tracing requires calculating precise collision (intersection) points between moving light rays and every object in a scene. Attempting to simulate light rays bouncing off a scene populated with tens of thousands of line segments using a brute-force $O(N)$ approach will inevitably result in a Time Limit Exceeded (TLE) error. To overcome this, you will implement a **QuadTree** to spatially partition the 2D environment, thereby significantly accelerating your ray tracing simulation.

Spatial Partitioning:

The core concept behind spatial partitioning is analogous to a Binary Search Tree (BST). Just as a BST divides a sorted array to reduce search time from $O(N)$ to $O(\log N)$, spatial partitioning data structures divide a 2D or 3D coordinate space to quickly isolate relevant geographic areas. There are several ways to partition space, differing primarily in how they define their splitting planes:

- **BSP Tree (Binary Space Partitioning):** Divides space using arbitrary planes (or lines in 2D), often aligned with the geometry itself.
- **KD-Tree (K-Dimensional Tree):** Recursively splits space along alternating coordinate axes (e.g., split by X, then by Y), adapting well to unevenly distributed points.
- **QuadTree (for 2D) / Octree (for 3D):** The simplest and most intuitive approach. It recursively divides a square into four equal-sized quadrants. For this assignment, we will use a **QuadTree** due to its straightforward implementation and excellent performance for general 2D space partitioning.

A QuadTree accelerates this process by organizing the 2D space into a hierarchical structure of nested squares. In a standard brute-force approach, determining the intersection point of a ray requires testing the ray against every single line segment in the entire scene. If there are 100,000 segments, every ray bounce demands 100,000 calculations ($O(N)$ time complexity). A QuadTree accelerates this process through hierarchical spatial partitioning, allowing the algorithm to quickly eliminate large regions of space to avoid unnecessary intersection tests. The key principles of this acceleration are as follows:

- **Space Division:** The 2D scene is initially wrapped in a large bounding box (the root node). As line segments are inserted, any box that exceeds a predefined segment capacity (e.g., 4 segments) subdivides into four smaller sub-boxes (NW, NE, SW, SE), distributing the segments among them.

- **Fast Rejection:** When a ray is fired, the algorithm checks for intersections against the QuadTree's bounding boxes. If a ray's path does not intersect a particular box, the algorithm instantly discards all line segments within that entire branch of the tree.
- **Localized Testing:** The ray only traverses the specific tree nodes that lie directly in its path. Once it reaches a leaf node, it performs the expensive ray-segment intersection math only on the few segments contained within that small area. By skipping vast empty spaces, this reduces intersection calculations from $O(N)$ to roughly $O(\log N)$.

C++ Implementation Framework

Below is a C++ structural framework illustrating the core data structures and algorithmic logic for building and querying the QuadTree. You can refer to this as a starting point for your implementation.

Data Structures & Helper Functions

The following structures define the foundational elements of our ray tracing and spatial partitioning mechanism. A Ray is defined by its origin and direction. An Intersection records whether a hit occurred, where it happened, and which object was hit. The QuadTreeNode represents a bounding box in 2D space, storing pointers to its four potential children and a list of objects contained within its boundaries. Additionally, a geometric helper function, `getIntersectParam`, is provided to calculate the intersection parameter between two infinite lines. You will need to build upon this logic to verify if the intersection point falls within the valid boundaries of your specific line segments.

```

1 struct Ray {
2     vec2 origin; // the starting point for testing intersections
3     vec2 direction; // the normalized direction vector of the ray
4 };
5
6 struct Intersection {
7     bool hit; // true if the ray hits an object
8     vec2 hit_point; // the coordinate of the intersection
9     Object* hit_object; // pointer to the object that was hit. For calculating
10    reflections.
11
12    Intersection(bool h) : hit(h) {}
13 };
14
15 // Define indices for the 4 quadrants
16 const int NE = 0, NW = 1, SE = 2, SW = 3;
17
18 struct QuadTreeNode {
19     // The spatial boundary of this node
20     vec2 min_bound;
21     vec2 max_bound;
22
23     // Array of 4 pointers to child nodes. (All nullptr if this is a Leaf node)
24     QuadTreeNode* children[4];
25
26     // Vector of object pointers contained within this node.
27     // (Internal nodes will clear this vector after distributing objects to leaves)
28     std::vector<Object*> objects;
29
30     QuadTreeNode(vec2 min_b, vec2 max_b) {
31         min_bound = min_b;
32         max_bound = max_b;
33         for (int i = 0; i < 4; ++i) children[i] = nullptr;
34     }
35 };

```

```

33     }
34 };
35
36 // Calculates the intersection parameters for lines extending through AB and CD.
37 // Returns the parameter 't' for representing the intersection point along AB.
38 float getIntersectParam(vec2 A, vec2 B, vec2 C, vec2 D) {
39     vec2 r = B - A;
40     vec2 s = D - C;
41
42     // 2D Cross product of direction vectors
43     float denom = r.x * s.y - r.y * s.x;
44
45     // If the denominator is close to 0, the lines are parallel or collinear
46     if (std::abs(denom) < 1e-6f) {
47         return std::numeric_limits<float>::max();
48     }
49     vec2 diff = C - A;
50
51     // Calculate and return parameters t
52     float t = (diff.x * s.y - diff.y * s.x) / denom;
53     return t;
54 }

```

Listing 1: Data Structures: Ray, Intersection, and QuadTreeNode

Building the QuadTree

The `build_tree` function recursively constructs the QuadTree. It starts by checking if the current node exceeds the maximum allowed object capacity and hasn't reached the maximum depth limit. If subdivision is needed, it computes the center of the node's bounding box and creates four child nodes representing the NW, NE, SW, and SE quadrants. The objects in the current node are then distributed into these children based on bounding box intersection tests. Finally, since internal nodes do not directly store objects, the current node's object list is cleared before recursively building the tree for its populated children.

```

1 // max_capacity: Max objects per node before subdivision
2 // max_depth: the maximum allowed depth of the tree to prevent infinite subdivision
3 // node: current QuadTree node being processed
4 // current_depth: the current depth in the tree (starting at 0 for the root)
5 void build_tree(QuadTreeNode* node, int current_depth, int max_capacity, int
    max_depth) {
6     // 1. Termination Check
7     if (node->objects.size() <= max_capacity || current_depth >= max_depth) {
8         return; // Node remains a Leaf
9     }
10
11     // 2. Subdivide Space
12     vec2 center = (node->min_bound + node->max_bound) * 0.5f;
13     node->children[NE] = new QuadTreeNode(/* ... NE bounds ... */);
14     node->children[NW] = new QuadTreeNode(/* ... NW bounds ... */);
15     node->children[SE] = new QuadTreeNode(/* ... SE bounds ... */);
16     node->children[SW] = new QuadTreeNode(/* ... SW bounds ... */);
17
18     // 3. Distribute Objects to Children
19     for (Object* obj : node->objects) {
20         for (int i = 0; i < 4; ++i) {
21             // Check if the object intersects with the child's bounding box
22             if (isIntersecting(obj, node->children[i]->min_bound, node->children[i]
                ->max_bound)) {
23                 node->children[i]->objects.push_back(obj);
24             }
25         }
26     }
27 }

```

```

26     }
27
28     // 4. Clean up and Recurse
29     // Internal nodes do not hold objects directly, only Leaf nodes do.
30     node->objects.clear();
31
32     for (int i = 0; i < 4; ++i) {
33         if (!node->children[i]->objects.empty()) {
34             build_tree(node->children[i], current_depth + 1, max_capacity,
35                 max_depth);
36         }
37     }

```

Listing 2: Building the QuadTree

Tracing the Ray

The `trace_ray` function finds the closest intersection of a given ray with the objects in the QuadTree. Upon reaching a leaf node, it performs the mathematically precise intersection tests against only the objects located in that specific region. For internal nodes, it first sorts the child regions by their entry distance relative to the ray. It then traverses these child nodes sequentially from nearest to farthest, returning the first valid intersection it finds to aggressively prune unnecessary calculations.

```

1 // node: current QuadTree node being traversed
2 // ray: the ray being traced
3 Intersection trace_ray(QuadTreeNode* node, Ray ray) {
4     // 1. Traversal Logic
5     // If this is a Leaf Node (no children)
6     if (node->children[0] == nullptr) {
7         // Perform exact Ray-Segment intersection math only for local objects
8         return FindClosestIntersection(ray, node->objects);
9     }
10
11     // If this is an Internal Node
12     // Sort child nodes based on the entry distance
13     std::vector<HitNode> hit_children = SortChildrenByDistance(ray, node);
14
15     // Recursively check children from nearest to farthest
16     for (HitNode hit : hit_children) {
17         Intersection isect = trace_ray(hit.node, ray);
18         if (isect.hit) {
19             return isect; // Stop traversal and return the closest valid hit
20         }
21     }
22
23     // 2. Reflection (Usually handled outside the recursive tree query)
24     // If a valid intersection is found, calculate the reflection vector
25
26     return Intersection(false); // Ray hit nothing in this node
27 }

```

Listing 3: Tracing the Ray

Example

The figure above illustrates a 2D ray tracing simulation accelerated by a QuadTree.

- **Black Lines:** Represent the line segments (reflective mirrors) in the environment.

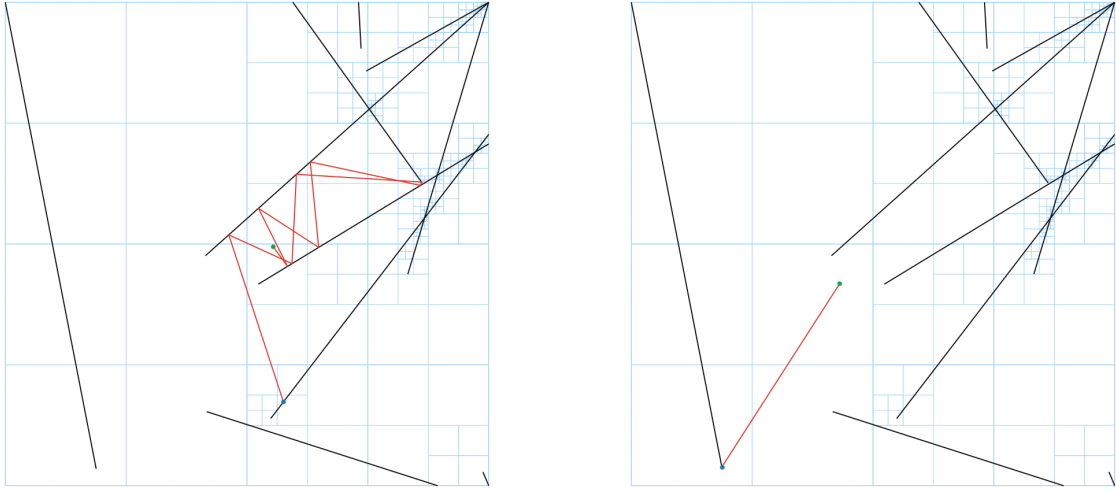


Figure 1: Visualization of a 2D ray tracing simulation accelerated by a QuadTree (configured with `MAX_CAPACITY = 1` and `MAX_DEPTH = 8`).

- **Light Blue Squares:** Represent the bounding boxes of the QuadTree nodes. In this specific example, the tree is configured to recursively subdivide whenever a node contains 1 or more line segments (up to a maximum depth). As a result, dense clusters of segments are heavily subdivided, while empty spaces remain as large, un-divided blocks.
- **Red Line:** Traces the path of a single ray. It originates at the starting position (green dot), bounces off several line segments, and eventually stops at its final position (blue dot) when reaching the maximum bounce depth.

Acceleration in Action: Notice the large empty light blue squares. When the ray travels through these areas, the algorithm instantly knows there are no line segments present. The ray can safely “skip” these massive regions without performing any mathematical intersection tests, only doing the heavy calculations when entering the smaller squares that actually contain the black lines.

Input

- The first line contains an integer N , representing the total number of line segments (mirrors) in the scene.
- The second line contains an integer `MAX_CAPACITY`, representing the maximum number of segments allowed in a single QuadTree node. If a node exceeds this threshold and has not yet reached the depth limit, it must be subdivided into four child nodes.
- The third line contains an integer `MAX_DEPTH`, representing the maximum allowed depth of the QuadTree. This prevents infinite subdivision in extremely densely packed areas.
- The next N lines each contain four floating-point numbers: x_1 y_1 x_2 y_2 , representing the start and end coordinates of a segment.
- The next line contains an integer B , representing the maximum recursion depth (max bounces) for the rays.
- The next line contains an integer R , representing the number of rays to trace.

- The next line contains an integer V , representing the visualization toggle (1 for ON, 0 for OFF).
- The next R lines each contain four floating-point numbers: $ox\ oy\ dx\ dy$, representing the initial origin (ox, oy) and direction vector (dx, dy) of a ray.

Constraints

- Number of line segments N : $1 \leq N \leq 100,000$.
- QuadTree capacity threshold $MAX_CAPACITY$: $1 \leq MAX_CAPACITY \leq 100$.
- QuadTree depth limit MAX_DEPTH : $1 \leq MAX_DEPTH \leq 20$.
- Maximum ray depth B : $1 \leq B \leq 100$.
- Number of rays R : $1 \leq R \leq 100$.
- Visualization toggle $V \in \{0, 1\}$.
- All coordinates are within the range $[-1000.0, 1000.0]$.

Output

For each ray, output its path sequentially to the terminal. The format must begin with `Ray [index]:`, followed by the origin point and each subsequent intersection point separated by commas `(,)`. You should round all coordinates to 2 decimal places for readability.

Handling Infinite Escapes:

If a ray does not collide with any object (i.e., it escapes the environment and extends to infinity), the path sequence in the terminal output must terminate at the last valid intersection point (or the origin, if no objects were hit). In your visualization (SVG), you do not need to render the final segment that extends toward infinity. Only the segments connecting the origin to valid intersection points should be drawn.

Floating-Point Precision & Accuracy Tolerance:

In practical ray tracing applications, rendering relies on stochastic sampling—firing multiple rays in random directions to approximate global illumination. Due to this statistical nature, absolute deterministic precision is not strictly necessary for every single ray. To emulate this, our test cases will evaluate your algorithm by shooting dozens of rays from the same origin. Your program is only required to correctly compute the paths for at least 95% of the rays to pass the test. Consequently, you do not need to worry about rare, extreme edge cases caused by floating-point inaccuracies. It is perfectly fine, and in fact highly encouraged, to use a small tolerance value (ϵ) during floating-point comparisons to avoid common artifacts like self-intersections.

Visualization Image Generation:

- If $V = 1$, your program must additionally generate an image file (e.g., an `.svg` file) for each ray, named `ray_path_<index>.svg`. **Similar to the provided example figure, this image must clearly depict the following three elements:**
 1. The spatial partition boundaries (the QuadTree nodes' bounding boxes).
 2. All objects in the environment (the reflective line segments).
 3. The final calculated ray path starting from its origin.

This visualization feature is primarily intended for debugging and correctness verification on smaller test cases.

- If $V = 0$, your program should **NOT** generate any image files; it should only print the text output to the terminal. This mode is reserved for massive stress tests (e.g., $N = 100,000$), where file I/O overhead would otherwise mask the CPU acceleration benefits achieved by your QuadTree structure.

Sample Input

```

1 4
2 2
3 2
4 -50.00 -50.00 50.00 -50.00
5 50.00 -50.00 50.00 50.00
6 50.00 50.00 -50.00 50.00
7 -50.00 50.00 -50.00 -50.00
8 5
9 1
10 1
11 0.00 0.00 1.00 1.00

```

Sample Output

```

1 Ray 0: (0.00,0.00) ,(50.00,50.00) ,(-50.00,50.00) ,(-50.00,-50.00)
      ,(50.00,-50.00) ,(50.00,50.00)

```

(Note: Because $V = 1$ in this sample, the program should also output a file named `ray_path_0.svg`)

2. Submission Guidelines

Please organize your code and files according to the following directory structure and compress them into a single `.zip` archive before uploading.

- **Single Submission Archive (.zip):** Place your main function code (`main.cpp`), all custom `.cpp` and `.h` files, and any other files required for compilation into a specific root folder. Compress this folder into a `.zip` file. (Note: If you organize your source codes into multiple subdirectories, you must include a `Makefile` in the root of your folder.)
- **The build Directory:** You must create a subfolder named `build` inside your root directory. This folder must contain your compiled executable file (`.exe`). Any required dependencies (e.g., `.dll` files) should also be placed in this folder.

Naming Example & Directory Structure: Zip File Name: `1142DS_<Student ID>_a5.zip` (e.g., `1142DS_B112345678_a5.zip`) Your `.zip` archive must follow this file structure:

```

1142DS_B123456789_a5.zip/
├── build/      # Must contain executable
│   ├── 1142DS_B123456789_a5.exe
│   └── ...    # dependencies(.dll)
├── main.cpp
└── ...      # And any other .cpp/.h/Makefile files

```

3. Grading Policies

- **Coding Style (10 points):** Assessed based on code readability, consistent indentation, and meaningful naming conventions. Arbitrary or meaningless variable names (e.g., `a`, `b`, `c`, `aa`, `bb`, `cc`) are strictly prohibited and will result in point deductions.
- **Comments (5 points):** Assessed based on brief and clear explanations of your function purposes and the overall processing flow. Focus on explaining the core logic. Do not add trivial or redundant comments for basic C++ syntax (e.g., commenting on `#include` or `using namespace std`). Additionally, every source file must include a file header specifying the author (including student ID), the date of writing (e.g., Mar. 22, 2026), and a description of the program's functionality and usage.
- **Correctness & Performance (35 points):** Points are awarded based on the percentage of test cases successfully passed. Your program must correctly trace at least 95% of the rays. Additionally, we will use `Measure-Command` in PowerShell to verify that your QuadTree implementation successfully accelerates the ray tracing process compared to a brute-force approach.

4. Allowed Libraries

For this assignment, you are allowed to use any standard or third-party libraries. It is highly recommended to use `std::vector` to manage input data and to use existing mathematical libraries for vector calculations. For example, our reference implementation utilizes the OpenGL Mathematics library (GLM, <https://github.com/g-truc/glm>).